

Bomberman

Projectverslag — Advanced Programming 2025-2026

Anna Voronovska

Studentennummer: 20240523

Github repo: https://github.com/AnnaVoronovska/Bomberman_game

CI-status: https://dl.circleci.com/status-badge/redirect/gh/AnnaVoronovska/Bomberman_game/tree/main

Projectoverzicht

Dit is mijn implementatie van Bomberman in C++ met SFML voor Advanced Programming. Net als de opgave vraagt, is de code opgesplitst in twee delen: een logic-library zonder SFML (alle spelregels) en een representatielaag die met SFML tekent en geluid afspeelt. Die scheiding komt terug in bijna elke keuze die ik hieronder bespreek.

Architectuur

De logic-library (`bomberman_logic`) bevat Core (Vec2, Camera, de singletons Random en Stopwatch), Observer, Entities (Wall, PowerUp, Door, Bomb, Character), World, Score, Command en Visitor. Deze library heeft geen SFML nodig om te compileren — dat kan je zelf checken in `CMakeLists.txt`, de `sfml-libraries` worden pas gelinkt bij het `bomberman-executable`, niet bij `bomberman_logic`.

De representatielaag (`src/frontend`) bevat Views (WallView, PowerUpView, DoorView, BombView, CharacterView), ConcreteFactory, AudioManager, Game en main. `Game.cpp` bevat de hoofdlus en tekent het startscherm/HUD/gameover-scherm, maar bevat zelf geen spellogica: het vertaalt enkel toetsaanslagen naar Commands die op de World worden uitgevoerd.

Eén bewuste afwijking van het voorbeeld-diagram uit de opgave: bij mij zit Camera in `Core.h`, dus in de logic-library, in plaats van bij Game Representation. Camera doet alleen coördinaten-wiskunde (van het genormaliseerde $[-1,1]$ -systeem naar pixels) en gebruikt nergens SFML, dus vond ik het logischer om hem bij de logica te zetten in plaats van bij de representatie.

Verplichte design patterns

- **Model-View-Controller:** World is de controller en beheert alle entiteiten en hun interacties (botsingen, explosies, win-conditie), maar tekent zelf niets. Elke entiteit (Wall, Bomb, PowerUp, Character, Door) is een Model met enkel data en regels. De bijhorende Views (WallView, BombView, PowerUpView, CharacterView, DoorView) weten hoe iets getekend wordt, maar bevatten geen spellogica.
- **Observer:** elk Model erft van Subject en kan observers hebben. Zodra ConcreteFactory een entiteit aanmaakt, koppelt hij daar meteen de View aan (en indien nodig de AudioManager) via `attach()`. De logica zelf weet niets van SFML of geluid — ze roept gewoon `notify(Event{...})` op en elke observer reageert op zijn eigen manier. Score is ook een observer: die luistert naar dezelfde events (BlockDestroyed, PowerUpCollected, EnemyKilled, PlayerWon, PlayerDied, ...) om de score bij te werken, volledig los van de tekencode.
- **Abstract Factory:** AbstractFactory (in de logic-library) beschrijft `createCharacter/createWall/createBomb/createPowerUp/createDoor` zonder te weten wat een SFML-sprite is. ConcreteFactory (in de representatie) maakt het Model aan, hangt er een View en eventueel de AudioManager aan, en geeft het Model terug. World kent enkel de abstracte interface.

- **Singleton:** Random en Stopwatch zijn allebei een singleton met een static instance()-methode. Random wordt gebruikt voor alles wat kans-gebaseerd is (muurgeneratie, power-ups, kans, welke breekbare muur de deur verbergt, bot-gedrag). Stopwatch berekent deltaTime via std::chrono, geen sf::Clock, zoals gevraagd.

Extra design patterns (bonus)

- **Visitor:** EntityVisitor kan over alle vijf entiteitstypes 'dispatchen' zonder ooit een dynamic_cast te gebruiken. Ik gebruik dit op twee plaatsen: EntityLabelVisitor bouwt een leesbare beschrijving van wat er op een tegel staat (gebruikt voor een debug-overlay in-game), en EntityScoreValueVisitor geeft aan hoeveel een entiteit 'waard' is. Dat laatste wordt niet gebruikt om de echte score te berekenen (dat doet Score al via Observer), maar is er vooral om te laten zien dat het patroon ook voor iets anders dan labels bruikbaar is.
- **Command:** speler-input (bewegen, bom plaatsen) wordt niet rechtstreeks als functie-aanroep naar World gestuurd, maar verpakt in een MoveCommand of PlaceBombCommand. Game.cpp maakt op basis van de ingedrukte toets een Command aan en voert die uit op de World. Dit ontkoppelt de bron van input (nu het toetsenbord) van de logica die de actie uitvoert. CommandHistory houdt bij welke commands zijn uitgevoerd; dat is geen vereiste van het patroon maar wel handig gebleken tijdens het debuggen van bot-gedrag.
- naast Visitor en Command implementeer ik ook 10 power-ups in plaats van de vereiste 3, een levens-systeem in plaats van instant-death, geluid en achtergrondmuziek voor alle belangrijke events, en een victory-animatie die afspeelt zodra de speler de deur op het laatste level bereikt (nu 2, bewust beperkt gehouden om het overzichtelijk te houden) en het spel wint — allemaal functionele extensies bovenop de kernvereisten van de opgave.

Gameplay & features

Een korte, praktische uitleg van de spelregels en besturing voor de speler is te vinden in README.md.

Startscherm & scores:

Bij het opstarten toont het spel een startscherm met de top 5 scores en een Play-knop. Klik je daarop, dan wordt het scherm gewist en start het spel. Scores worden bijgehouden in Score (via Observer, zie hierboven) en opgeslagen in highscores.txt, zodat ze blijven staan tussen verschillende keren dat je het spel opstart. Puntenverdeling: +10 per vernielde breekbare muur, +25 per power-up, +150 per gedode vijand, +500 bij winnen, -50 bij verliezen (nooit onder 0).

Arena:

De arena is 13x11 tegels, met een border van onverwoestbare muren en om de andere rij/kolom een onverwoestbare pilaar, net als in figuur 1 van de opgave. De overige tegels zijn met 80% kans een breekbare muur en anders lege lucht. De vier hoeken (speler + 3 bots) hebben altijd een vrije L-vormige spawnzone, zodat niemand meteen ingesloten start. Eén willekeurig gekozen breekbare muur verbergt een deur: zodra die muur ontploft, verschijnt de deur. Loop je erover als speler, dan ga je naar de volgende stage (nieuwe arena, zelfde speler/score/power-ups). Er zijn 2 levels; op level 2 win je het spel in plaats van door te gaan.

Besturing & botsing:

De pijltjestoetsen sturen de speler continu (geen discrete stappen), en botsing met muren/bommen gebeurt via AABB (overlappende rechthoeken), zonder SFML-utilities, zoals de opgave vraagt. De speler start linksboven.

Bommen en explosies:

Spatie plaatst een bom op de tegel van de speler. Zolang je erop staat kan je er nog doorheen lopen, maar zodra je die tegel verlaat niet meer. Na 2 seconden ontploft de bom in een kruisvorm, stopt bij onverwoestbare muren, en breekt maximaal één breekbare muur per richting. Explosies kunnen andere bommen in hun bereik triggeren (kettingreactie) en vernielen power-ups binnen bereik. Iedereen (speler of bot) die geraakt wordt door een explosie verliest een leven.

Levens in plaats van instant-death:

De opgave zegt dat een geraakt character 'onmiddellijk sterft'. Ik heb hier bewust van afgeweken en een levens-systeem toegevoegd: elk character (speler en bots) begint met 3 levens, en na een treffer is het character eventjes onkwetsbaar zodat je niet in dezelfde explosie meerdere levens verliest. Pas bij 0 levens sterft het character echt. Dat maakt het spel wat minder frustrerend zonder de kernregel (contact met een explosie is gevaarlijk) te veranderen.

Power-ups

Breekbare muren hebben een kans (25%) om bij het breken een power-up te spawnen. De opgave vraagt er minstens 3 (Fire, Extra Bomb, Skates); ik heb er in totaal 10 geïmplementeerd, waarvan de helft negatief is, voor wat extra spanning bij het breken van muren.

Power-up	Effect
Fire	bomradius +1
Extra Bomb	max. aantal bommen tegelijk +1
Skates	snellheid +0.15
Poison	snellheid -0.1 (min. 0.1)
Star	bonuspunten, geen stat-wijziging
Shield	bomradius +2
Curse	max. bommen -1 (min. 1)
Slow	snellheid -0.15 (min. 0.1)
Freeze	snellheid +0.25
Skull	bomradius -1 (min. 1)

Bot-AI

De drie bots draaien elke tick door een prioriteitenlijst, zoals de opgave beschrijft:

- Overleven: als een bot binnen het bereik van een bom staat, zoekt hij via BFS de dichtstbijzijnde veilige tegel en vlucht daarheen.
- Hebzucht: is er geen direct gevaar, dan loopt de bot via pathfinding (ook BFS) naar de dichtstbijzijnde power-up.
- Agressie: staat de bot naast een breekbare muur of een andere speler, dan plaatst hij een bom — maar alleen als hij op voorhand al weet dat hij daarna nog een veilige vluchtroute heeft. Zo niet, doet hij het niet.
- Zoeken: is er niets in de buurt, dan loopt de bot naar de dichtstbijzijnde breekbare muur om zelf actief speelruimte te creëren.
- Fallback: is er niets meer te doen, dan loopt de bot willekeurig rond.

Ik heb dit met echte BFS-pathfinding gedaan in plaats van simpele dx/dy-vergelijkingen, omdat een bot anders tegen een muur of pilaar kon blijven 'plakken' zodra zijn eerste keuze-richting toevallig geblokkeerd was. Met BFS kiest de bot altijd de eerste stap van een pad dat ook echt bestaat, of concludeert eerlijk dat er geen pad is en valt terug op de volgende prioriteit.

Animaties

Elk character heeft een loop-animatie in 4 richtingen (SFML-spritesheet, 4 frames per richting, rechts is een gespiegelde versie van links). Bommen hebben een 'ademende' puls die sneller gaat naarmate de lont korter wordt, en spelen bij ontploffing een korte groei-animatie af. Sterft een character, dan speelt er een animatie waarbij het personage ronddraait, krimpt en vervaagt in plaats van gewoon te verdwijnen. Wint de speler (deur bereikt op het laatste level), dan speelt er een korte 'huppel'-animatie op de eindpositie.

Geluid

AudioManager is ook gewoon een Observer, op dezelfde manier aangesloten als de Views: hij luistert naar dezelfde events (bom ontploft, power-up opgeraapt, character sterft, speler wint, ...) en speelt het bijhorende geluid af. Er is ook achtergrondmuziek. Ontbrekende geluidsbestanden crashen het spel niet, ze worden gewoon overgeslagen met een waarschuwing.

Technische kwaliteit

- **Smart pointers:** overal `shared_ptr` voor entiteiten (eigenaarschap gedeeld tussen World en de Views die er via Observer op geabonneerd zijn) en `weak_ptr` waar geen eigenaarschap hoort te zijn, bijvoorbeeld de owner van een Bomb of de observer-lijst in Subject zelf. Geen raw pointers.
- **Geen dynamic_cast:** dankzij het Visitor-patroon moet er nergens gecast worden om te weten met welk entiteitstype je te maken hebt.
- **const/override:** consequent gebruikt, bijvoorbeeld alle getters zijn `const` en elke override van een virtuele functie is met `override` gemarkeerd.
- **Exception handling:** het laden van de spritesheet en het lettertype gebeurt met `try/catch` in `main/Game`, zodat een ontbrekend bestand een duidelijke foutmelding geeft in plaats van een crash.
- **Unit tests:** ongeveer 80 tests met GoogleTest, verspreid over entiteiten (Wall, PowerUp, Bomb, Character), Score, Command en Visitor. De tests compileren de logic-library rechtstreeks vanuit de bron, los van het hoofdproject, zodat je ze kan draaien zonder SFML te installeren.
- **Valgrind:** getest op memory leaks, geen gevonden.
- **CI:** een CircleCI-configuratie (`.circleci/config.yml`) bouwt het project bij elke push op een schone Ubuntu 24.04-image.
- **Code style:** consequent geformatteerd met clang-format volgens de opgegeven configuratie (`.clang-format` in de repo).

DOXYGEN Documentatie

De code is gedocumenteerd met Doxygen: elke class en publieke methode heeft een `@brief`-commentaar die uitlegt wat ze doet. De documentatie is te vinden in de map:

`docs/doxygen/html/index.html` van de GitHub-repo, en bevat ook class-overzichten, overervingsdiagrammen en collaboration-diagrammen per class.

Classes Diagram

Dit diagram (automatisch gegenereerd met Doxygen) toont de overervingsstructuur van alle classes in het project. De scheiding tussen de logic-library (namespace `bomberman`) en de representatielaag (Views, ConcreteFactory, Game) is duidelijk zichtbaar, net als de verplichte en extra design patterns (Observer, Visitor, Command, Abstract Factory).

Bomberman

C++/SFML Bomberman - Advanced Programming project

[Main Page](#)

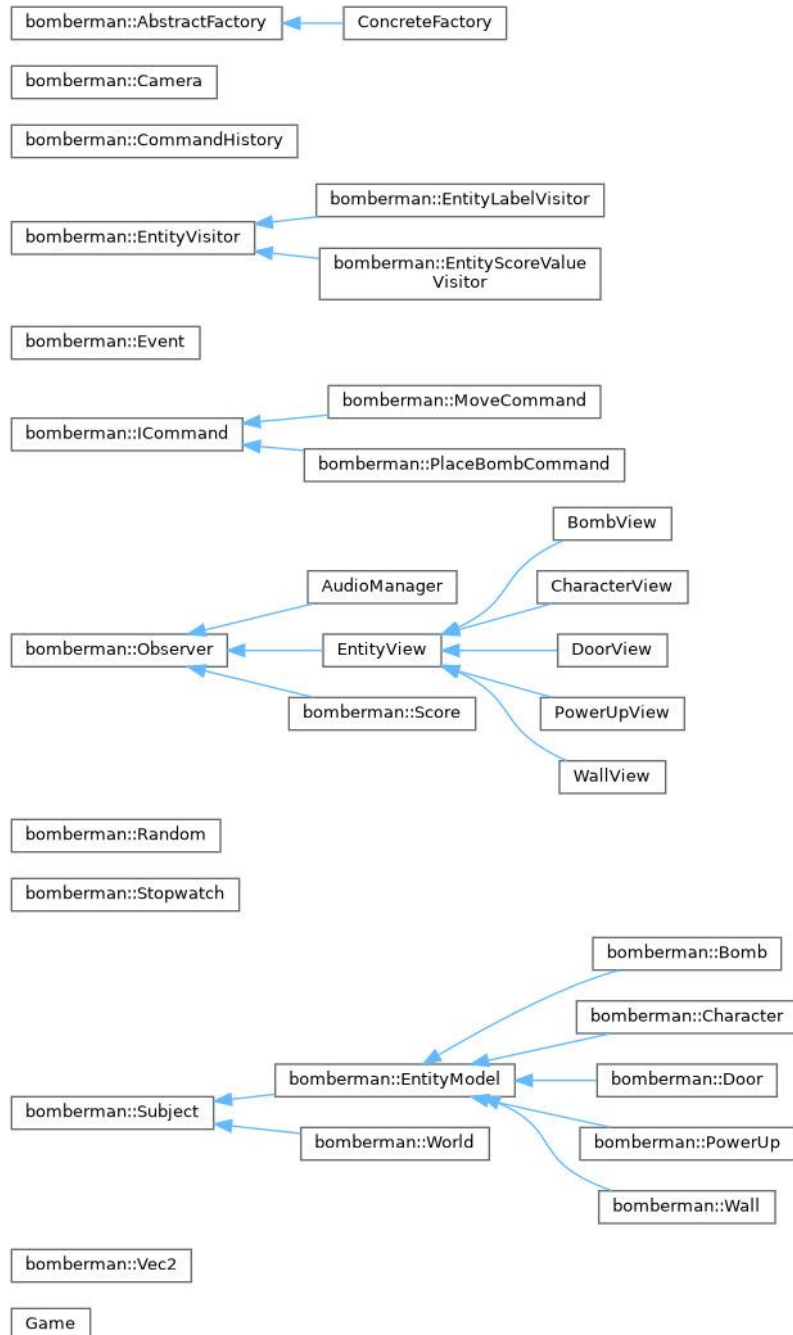
[Namespaces ▾](#)

[Classes ▾](#)

[Files ▾](#)

Class Hierarchy

[Go to the textual class hierarchy](#)



Reflectie

Het lastigste deel was de bot-AI: mijn eerste versie koos een richting puur op basis van dx/dy tussen bot en doel, en dat liep vast zodra er een muur precies tussenin stond. Overschakelen naar echte BFS-pathfinding (zowel voor vluchten als voor doelen zoeken) heeft dat opgelost, en heeft ook de 'bot

loopt zichzelf in de lucht'-bug weggewerkt die ontstond doordat een bot na het vluchten soms meteen weer een gevaarlijke tegel in werd gestuurd.

Verbeterpunten die ik nog zie: Ik zou graag nog wat meer variatie in bot-persoonlijkheden toevoegen, bijvoorbeeld de ene loopt heel snel de andere zet bommen met bereik 2 tegels, enzovoort.

Ook zou ik het aantal levels (nu 2, bewust beperkt gehouden om het overzichtelijk te houden) makkelijk kunnen uitbreiden, aangezien MAX_LEVEL een losstaande constante is.

Leerpunt: ik ben blijven verbeteren nadat de basis al werkte, wat wel leuk was om te doen maar ook wat tijd heeft gekost.